

ICT 1306

OBJECT ORIENTED PROGRAMMING

5.OPERATOR OVERLOADING

TC Irugalbandara
KHA Hettige

Operator Overloading

- Operator overloading refers to giving the normal C++ operators, such as +, *, <=, and +=, additional meanings when they are applied to user-defined data types.

Ex: int a,b,c;

c=a+b; // What is the output?

Distance d1,d2,d3;

d3=d1+d2; // What should be the output???

Overloading Unary Operators

- Unary operators act on only one operand.
- Ex: increment operator(++), decrement operator(--), minus (-)
- Implement a class named "Counter" with a type int data member initialize to zero. Write a member function to increment the value by 1.

Structure of the Operator Function

```
class class_name
{
    .....
public:
    return_type operator sign (argument/s)
    {
        .....
    }
    .....
};
```

Overload Increment Operator

```
#include <iostream>
using namespace std;
////////////////////////////////////
class Counter
{
private:
    unsigned int count;           //count
public:
    Counter() : count(0)          //constructor
    { }
    unsigned int get_count()       //return count
    { return count; }
    void operator ++ ()            //increment (prefix)
    {
        ++count;
    }
};
```

```
int main()
{
    Counter c1, c2;                //define and initialize

    cout << "\nc1=" << c1.get_count(); //display
    cout << "\nc2=" << c2.get_count();

    ++c1;                          //increment c1
    ++c2;                          //increment c2
    ++c2;                          //increment c2

    cout << "\nc1=" << c1.get_count(); //display again
    cout << "\nc2=" << c2.get_count() << endl;
    return 0;
}
```

Key Word: operator

- The only way to distinguish between overloaded operators and normal operator is by looking at the data type of their operands.
- If the operand is a basic type such as an int then the compiler will use its built-in routine to increment an int. But if the operand is a variable of a user defined type, the compiler will know to use our user-written operator++() instead.
- Ex : `int a,b,c;`
`c=a+b;`
`Distance d1,d2,d3;`
`d3=d1+d2;`

- `c1=++c;` what is the output???

```
void operator ++ ()
{
    ++count;
}
```

- Need to modify as

```
Counter operator ++ ()
{
    ++count;
    Counter temp;
    temp.count = count;
    return temp;
}
```

Temporary objects (Nameless)

```
class Counter
{
private:
    unsigned int count;
public:
    Counter() : count(0)
    { }
    Counter(int c) : count(c)
    { }
    unsigned int get_count()
    { return count; }
    Counter operator ++ ()
    {
        ++count;
        return Counter(count);
    }
};
```


- Postfix Increment Operator

```
Counter operator ++ (int)
{
    return Counter(count++);
}
```

- The only difference is the int in the parentheses.
- This int isn't really an argument, and it doesn't mean integer. It's simply a signal to the compiler to create the postfix version of the operator.
- The designers of C++ are fond of recycling existing operators and keywords to play multiple roles, and int is the one they chose to indicate postfix.

Overloading Binary Operators

- Arithmetic Operators : Addition(+) Operator

```
dist3.add_dist(dist1, dist2);
```



```
dist3 = dist1 + dist2;
```

- Declare the following in the class and define it.

```
Distance operator + ( Distance ) const;
```

```
Distance Distance::operator + (Distance d2) const //return sum
{
    int f = feet + d2.feet;           //add the feet
    float i = inches + d2.inches;     //add the inches
    if(i >= 12.0)                      //if total exceeds 12.0,
    {                                  //then decrease inches
        i -= 12.0;                    //by 12.0 and
        f++;                           //increase feet by 1
    }                                  //return a temporary Distance
    return Distance(f,i);              //initialized to sum
}
```

The argument on the left side of the operator (dist1 in this case) is the object of which the operator is a member. The object on the right side of the operator (dist2) must be furnished as an argument to the operator.

- We can generalize and say that an overloaded operator always requires one less argument than its number of operands, since one operand is the object of which the operator is a member. That's why unary operators require no arguments.
- **Multiple Overloading**
 - You could put multiple overloaded classes together in the same program, and C++ would still know how to interpret the + operator.
 - It selects the correct function to carry out the "addition" based on the type of operand.

Overloading Comparison Operators

```
class Distance                                //English Distance class
{
private:
    int feet;
    float inches;
public:
    //constructor (no args)
    Distance() : feet(0), inches(0.0)
    { }
    //constructor (two args)

    Distance(int ft, float in) : feet(ft), inches(in)
    { }
    void getdist()                            //get length from user
    {
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
    void showdist() const                     //display distance
    { cout << feet << "'-" << inches << "\""; }
    bool operator < (Distance) const;        //compare distances
};
```

```
bool Distance::operator < (Distance d2) const //return the sum
{
    float bf1 = feet + inches/12;
    float bf2 = d2.feet + d2.inches/12;
    return (bf1 < bf2) ? true : false;
}
```

Inside the main

```
if( dist1 < dist2 ) //overloaded '<' operator
    cout << "\ndist1 is less than dist2";
else
    cout << "\ndist1 is greater than (or equal to) dist2";
..
```

Arithmetic Assignment Operator Overloading

Declare inside the class as

```
void operator += ( Distance );
```

Define it

```
void Distance::operator += (Distance d2)
{
    feet += d2.feet;           //add the feet
    inches += d2.inches;       //add the inches
    if(inches >= 12.0)          //if total exceeds 12.0,
    {                           //then decrease inches
        inches -= 12.0;        //by 12.0 and
        feet++;                //increase feet
    }                           //by 1
}
```

Call it

```
dist1 += dist2;
```

Data Conversion

- The = operator will assign a value from one variable to another.
Ex: `int a=3,b=10;`
`a=b;`
- What If
`Distance dist1,dist2;`
`dist2 = dist1`
- When the value of one object is assigned to another of the same type, the values of all the member data items are simply copied into the new object. The compiler doesn't need any special instructions to use = for the assignment of user-defined objects such as Distance objects.

Conversions Between Basic Types

- If we write

```
intvar = floatvar;
```

- Compiler has its own routine and calls up when different data types are available with equal sign. such conversions are **implicit**
- Sometimes we want to force the compiler to convert one type to another. To do this we use the **cast operator**.

```
intvar = static_cast<int>(floatvar);
```

- This is a explicit conversion , such explicit conversions use the same built-in routines as implicit conversions.

Conversions Between Objects and Basic Types

- When we want to convert between user-defined data types and basic types, we can't rely on built-in conversion routines, since the compiler doesn't know anything about user-defined types.
- We need to explicitly say what to do.

```

class Distance                                //English Distance class
{
private:
    const float MTF;                          //meters to feet
    int feet;
    float inches;
public:
    //constructor (no args)
    Distance() : feet(0), inches(0.0), MTF(3.280833F)
    { }
    //constructor (one arg)
    Distance(float meters) : MTF(3.280833F)
    {
        //convert meters to Distance
        float fltfeet = MTF * meters; //convert to float feet
        feet = int(fltfeet);           //feet is integer part
        inches = 12*(fltfeet-feet);    //inches is what's left
    }
    //constructor (two args)
    Distance(int ft, float in) : feet(ft),
                                inches(in), MTF(3.280833F)
    { }
    void getdist()                //get length from user
    {
        cout << "\nEnter feet: "; cin >> feet;
        cout << "Enter inches: "; cin >> inches;
    }
    void showdist() const         //display distance
    { cout << feet << "\'-" << inches << '\\"'; }

    operator float() const       //conversion operator
    {
        //converts Distance to meters
        float fracfeet = inches/12; //convert the inches
        fracfeet += static_cast<float>(feet); //add the feet
        return fracfeet/MTF;       //convert to meters
    }
};

```

```
int main()
{
    float mtrs;
    Distance dist1 = 2.35F;          //uses 1-arg constructor to
                                     //convert meters to Distance
    cout << "\ndist1 = "; dist1.showdist();

    mtrs = static_cast<float>(dist1); //uses conversion operator
                                     //for Distance to meters
    cout << "\ndist1 = " << mtrs << " meters\n";

    Distance dist2(5, 10.25);        //uses 2-arg constructor

    mtrs = dist2;                    //also uses conversion op
    cout << "\ndist2 = " << mtrs << " meters\n";

    // dist2 = mtrs;                  //error, = won't convert
    return 0;
}
```

- Converting a user-defined type to a basic type requires a different approach than converting a basic type to a user-defined type.

From Basic to User-Defined

- Use a constructor with one argument. These are sometimes called conversion constructors

From User-Defined to Basic

- Need to create something called a conversion operator.
- Ex: This operator takes the value of the Distance object of which it is a member, converts it to a float value representing meters, and returns this value.
- This operator can be called with an explicit cast.
 - `mtrs = static_cast<float>(dist1);` or simply by
 - `mtrs = dist2;`

Conversions Between Objects of Different Classes

- The same two methods
 - a one-argument constructor or
 - a conversion operator.
- The choice depends on whether you want to put the conversion routine in the class declaration of the source object or of the destination object.

objecta = objectb;

The diagram illustrates the conversion of objectb to objecta. Two blue arrows point from the labels 'Destination Obj' and 'Source Obj' to the variables 'objecta' and 'objectb' respectively in the code snippet above. 'objecta' is the destination object and 'objectb' is the source object.

Destination Obj

Source Obj

Important

- The following operators cannot be overloaded:
 - the member access or dot operator (.),
 - the scope resolution operator (::),
 - the conditional operator (?:).
- You can't create new operators and try to overload them; only existing operators can be overloaded.

	<i>Routine in Destination</i>	<i>Routine in Source</i>
Basic to basic	(Built-In Conversion Operators)	
Basic to class	Constructor	N/A
Class to basic	N/A	Conversion operator
Class to class	Constructor	Conversion operator

Thank You !